

GUJARAT TECHNOLOGICAL UNIVERSITY**BE- SEMESTER-V (NEW) EXAMINATION – WINTER 2024****Subject Code:3150703****Date:17-12-2024****Subject Name:Analysis and Design of Algorithms****Time:10:30 AM TO 01:00 PM****Total Marks:70****Instructions:**

1. Attempt all questions.
2. Make suitable assumptions wherever necessary.
3. Figures to the right indicate full marks.
4. Simple and non-programmable scientific calculators are allowed.

- | | Marks |
|---|--------------|
| Q.1 (a) Define the following terms: | 03 |
| i) Function in mathematics ii) Linear inequalities | |
| (b) Explain the need of amortized notation. Explain with suitable example. | 04 |
| (c) Write an algorithm for insertion sort. Calculate the best, average and worst case complexity of it. | 07 |
|
Q.2 (a) Sort the following elements using counting sort. | 03 |
| 2, 5, 3, 0, 2, 3, 0, 3 | |
| (b) Sort the following elements using bucket sort. | 04 |
| 10, 21, 29, 41, 52 | |
| (c) Write the Master theorem. Solve following recurrence using it. | 07 |
| (i) $T(n) = 3T(n/2) + n^2$ | |
| (ii) $T(n) = 2T(n/2) + n \log n$ | |
| OR | |
| (c) Write binary search algorithm using recursion. Derive its recurrence relation. Calculate its complexity using master theorem. | 07 |
|
Q.3 (a) Explain the steps of greedy strategy for solving a problem. | 03 |
| (b) Write an algorithm for Max-Min problem using divide and conquer approach. Calculate its complexity. | 04 |
| (c) The matrices A(5X10), B(10X15), C(15X20), and D(20X25) are given. Solve the matrix chain multiplication problem using dynamic programming. | 07 |
| OR | |
| Q.3 (a) Enlist the general characteristics of greedy algorithms. | 03 |
| (b) Write an algorithm for matrix multiplication using divide and conquer approach. Calculate its complexity. | 04 |
| (c) For the given set of items and knapsack capacity = 5 kg, find the optimal solution for the 0/1 knapsack problem using dynamic programming. | 07 |

Item	Weight	Value
I1	2	3
I2	3	4
I3	4	5
I4	5	6

- | | |
|--|-----------|
| Q.4 (a) Explain DFS with suitable example. | 03 |
| (b) Enlist the advantages and disadvantages of dynamic programming. | 04 |

	(c)	Define minimum spanning tree. Explain Prim's algorithm with suitable example.	07
		OR	
Q.4	(a)	Explain BFS with suitable example.	03
	(b)	Define principal of optimality. Explain its use in Dynamic Programming Method.	04
	(c)	"A greedy strategy will work for fractional Knapsack problem but not for 0/1", is this true or false? Explain with suitable example.	07
Q.5	(a)	Define string matching problem. Define valid shift and invalid shift.	03
	(b)	Define the followings: i) Articulation point ii) Acyclic Directed Graph iii) Back Edge iv) Tree	04
	(c)	Explain P, NP, NP complete and NP-Hard problems. Give examples of each.	07
		OR	
Q.5	(a)	Explain Backtracking Method. Give solutions for 4-queens problem using backtracking method.	03
	(b)	Explain in-order, pre-order and post-order traversals of the graph.	04
	(c)	Write short note on the followings: i) Approximation algorithms ii) Randomized algorithms	07

Q.1 (a) Define the following terms:

i) Function in mathematics ii) Linear inequalities (3 mark)

i) Function in Mathematics:

A **function** is a relation between two sets in which **each element of the first set (domain) is associated with exactly one element of the second set (codomain)**. It is often denoted as $f: X \rightarrow Y$, where X is the domain and Y is the codomain.

Key Points:

- Every input has exactly **one output**.
- Functions can be represented as **equations, graphs, tables, or mappings**.
- Examples:
 - $f(x) = x^2$, where for each real number x , there is a unique value x^2 .
 - $f(x) = 3x + 2$, a linear function mapping real numbers to real numbers.

Purpose / Importance:

- Functions describe relationships between variables in **mathematics, physics, and engineering**.
- They help in modeling real-world problems and performing calculations efficiently.

ii) Linear Inequalities:

A **linear inequality** is an inequality involving a **linear expression** of one or more variables. It compares two expressions using symbols such as $<$, $\leq$, $>$, $\geq$.

General Form:

- For one variable: $ax + b < 0$ or $ax + b \geq 0$
- For two variables: $ax + by \leq c$ or $ax + by > c$

Key Points:

- The graph of a linear inequality in two variables is a **half-plane**, not just a line.
- Linear inequalities are used in **optimization problems, resource allocation, and decision-making**.

Example:

- $2x + 3y \leq 6$ represents all points (x, y) *represents all points* (x, y) *lying on or below the line* $2x + 3y = 6$.

(b) Explain the need of amortized notation. Explain with suitable example. (4 mark)

Definition / Introduction:

Amortized analysis is a method used in algorithm analysis to **average the running time of operations over a sequence of operations**, rather than considering the worst-case time of a single operation. The **amortized notation** gives a more realistic measure of the **overall performance** of an algorithm when multiple operations are performed consecutively.

Need of Amortized Notation

- Some operations may occasionally take a **long time** (worst-case), while most operations are fast.
- Worst-case analysis may **overestimate** the running time for practical usage.
- Amortized analysis provides a **smoothed or averaged cost** per operation.

- Useful in **data structures** like dynamic arrays, stacks, queues, and hash tables.

Key Points:

- Helps understand the **true cost** of a sequence of operations.
- Important in **designing efficient algorithms**.
- Ensures **guaranteed performance** over multiple operations rather than just single instances.

Example: Dynamic Array / Array Doubling

- Consider a **dynamic array** that doubles in size when it becomes full:
 - Inserting an element normally takes $O(1)$ time.
 - When the array is full, resizing and copying all elements takes $O(n)$ time.

Analysis using Amortized Notation:

- Suppose we insert n elements:
 - Total cost = cost of all individual insertions + resizing cost.
 - Resizing occurs at sizes 1, 2, 4, 8, ...
 - Total resizing cost $\leq 2n$
- **Amortized cost per insertion:** $\frac{O(n) + O(n)}{n} = O(1)$
- So, each insertion costs **$O(1)$ amortized**, even though occasional insertions may take $O(n)$.

(c) Write an algorithm for insertion sort. Calculate the best, average and worst case complexity of it. (7 mark)

Insertion Sort Algorithm

Introduction / Definition:

Insertion Sort is a simple sorting algorithm that builds the final sorted array one element at a time. It works similarly to the way people sort playing cards in their hands: by taking one element at a time and inserting it into its correct position among the already sorted elements. This algorithm is efficient for small datasets and nearly sorted arrays.

Detailed Explanation

- **Idea:** Divide the array into a sorted and unsorted part. Take each element from the unsorted part and insert it into the correct position in the sorted part.
 - **Process:**
 1. Start with the second element (index 1) as the key.
 2. Compare it with elements in the sorted part (left side).
 3. Shift all elements greater than the key to the right.
 4. Insert the key in the correct position.
 5. Repeat for all remaining elements.
-

Algorithm (Pseudocode)

InsertionSort($A[0..n-1]$):

 for $i = 1$ to $n-1$:

$key = A[i]$

$j = i - 1$

 while $j \geq 0$ and $A[j] > key$:

$A[j + 1] = A[j]$ // Shift element to the right

$$j = j - 1$$

$A[j + 1] = \text{key}$ // Insert key at correct position

Steps / Example

Example Array: [5, 2, 4, 6, 1, 3]

1. $i = 1$, $\text{key} = 2 \rightarrow$ Compare with 5 $\rightarrow$ Shift 5 $\rightarrow$ Insert 2 $\rightarrow$ [2, 5, 4, 6, 1, 3]
 2. $i = 2$, $\text{key} = 4 \rightarrow$ Compare with 5 $\rightarrow$ Shift 5 $\rightarrow$ Insert 4 $\rightarrow$ [2, 4, 5, 6, 1, 3]
 3. $i = 3$, $\text{key} = 6 \rightarrow$ Already in correct place $\rightarrow$ [2, 4, 5, 6, 1, 3]
 4. $i = 4$, $\text{key} = 1 \rightarrow$ Shift 6,5,4,2 $\rightarrow$ Insert 1 $\rightarrow$ [1, 2, 4, 5, 6, 3]
 5. $i = 5$, $\text{key} = 3 \rightarrow$ Shift 6,5,4 $\rightarrow$ Insert 3 $\rightarrow$ [1, 2, 3, 4, 5, 6]
-

Time Complexity

- **Best Case:** $O(n)$
 - Occurs when the array is already sorted. Only one comparison per element is needed.
- **Average Case:** $O(n^2)$
-
- On average, half of the sorted part needs to be shifted for each insertion.
- **Worst Case:** $O(n^2)$
- Occurs when the array is sorted in reverse order. Maximum comparisons and shifts are required.
- **Space Complexity:** $O(1)$ (in-place sorting)

- ---
- **Key Features**
- Simple and easy to implement
- Efficient for small or nearly sorted arrays
- Stable sorting algorithm (does not change the relative order of equal elements)
- ---
- **Advantages**
- Minimal memory usage (in-place)
- Performs well on small datasets or nearly sorted arrays
- **Disadvantages**
- Not suitable for large datasets due to quadratic time complexity
- Performance degrades for large, randomly ordered arrays
- ---
- **Applications**
- Used in small embedded systems with limited memory
- Sorting small datasets in educational examples
- Step in hybrid sorting algorithms like **Timsort**

Q.2 (a) Sort the following elements using counting sort. 2, 5, 3, 0, 2, 3, 0, 3 (3 mark)

Introduction / Definition:

Counting Sort is a non-comparison-based sorting algorithm used for sorting integers within a known range. It works by counting the number of occurrences of each distinct element in the input array and then calculating the positions of each element in the sorted

output. Counting Sort is efficient for small ranges of integers and is **stable**.

Steps of Counting Sort

1. Find the range of elements:

- Minimum = 0, Maximum = 5

2. Create a count array:

- Initialize $\text{count}[0..5] = [0,0,0,0,0,0]$

3. Count occurrences of each element:

- Input array: [2, 5, 3, 0, 2, 3, 0, 3]
- Count array after counting:
 - $0 \rightarrow 2$
 - $2 \rightarrow 2$
 - $3 \rightarrow 3$
 - $5 \rightarrow 1$
- So, $\text{count} = [2,0,2,3,0,1]$

4. Cumulative count (optional for stable sort):

- $\text{count}[0] = 2$
- $\text{count}[1] = 2+0 = 2$
- $\text{count}[2] = 2+2 = 4$
- $\text{count}[3] = 4+3 = 7$
- $\text{count}[4] = 7+0 = 7$
- $\text{count}[5] = 7+1 = 8$
- Cumulative count array: [2, 2, 4, 7, 7, 8]

5. Place elements in sorted order:

- Start from the last element for stability:
 - $3 \rightarrow \text{position } 6 \rightarrow \text{output } [, , , , , 3,]$
 - $0 \rightarrow \text{position } 1 \rightarrow \text{output } [0, , , , , ,]$
 - Continue for all elements

6. Sorted Output:

7. $[0, 0, 2, 2, 3, 3, 3, 5]$

Key Points

- **Time Complexity:** $O(n + k)$, where n = number of elements, k = range of elements
- **Space Complexity:** $O(n + k)$
- **Stable:** Yes, maintains relative order of equal elements
- Efficient when the range of integers is **not significantly larger than the number of elements**

(b) Sort the following elements using bucket sort. 10, 21, 29, 41, 52
(4 mark)

Bucket Sort

Definition / Introduction:

Bucket Sort is a sorting algorithm that distributes elements of an array into a number of **buckets**. Each bucket is then sorted individually, either using another sorting algorithm or recursively applying bucket sort. Finally, the sorted buckets are concatenated to form the sorted array. Bucket sort is efficient when input elements are **uniformly distributed** over a range.

Steps of Bucket Sort

1. Input Array:

2. [10, 21, 29, 41, 52]

3. Create Buckets:

- Choose the number of buckets based on the range of elements.
- Suppose we create **5 buckets** for simplicity.

4. Distribute Elements into Buckets:

- Bucket 1: 0–10 → [10]
- Bucket 2: 11–20 → []
- Bucket 3: 21–30 → [21, 29]
- Bucket 4: 31–40 → []
- Bucket 5: 41–50 → [41]
- Bucket 6: 51–60 → [52]

5. Sort Each Bucket (if necessary):

- Buckets with multiple elements: [21, 29] → already sorted
- Other buckets contain 1 element → already sorted

6. Concatenate Buckets:

- [10] + [] + [21, 29] + [] + [41] + [52]
- Sorted array:
- [10, 21, 29, 41, 52]

Key Points

- **Time Complexity:**

- Best case (uniform distribution) $\rightarrow O(n + k)$, where n = number of elements, k = number of buckets
- Worst case (all elements in one bucket) $\rightarrow O(n^2)$
- **Space Complexity:** $O(n + k)$
- **Stable Sorting Algorithm** (if stable sort used within buckets)
- Efficient for **floating-point numbers or integers within a known range**

(c) Write the Master theorem. Solve following recurrence using it.

(i) $T(n) = 3T(n/2) + n^2$ (ii) $T(n) = 2T(n/2) + n \log n$ (7 mark)

Master Theorem

Introduction / Definition:

The **Master Theorem** is a method used in computer science to determine the **time complexity of divide-and-conquer algorithms** expressed as recurrence relations of the form:

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

Where:

- $a \geq 1 \rightarrow$ number of subproblems
- $b > 1 \rightarrow$ factor by which problem size reduces
- $f(n) \rightarrow$ cost of dividing and combining subproblems

The theorem provides asymptotic bounds based on the comparison of $f(n)$ with $n^{\log_b a}$

Master Theorem Cases

1. **Case 1:** $f(n) = O(n^{\log_b a - \epsilon})$, $\epsilon > 0$
 $\Rightarrow T(n) = \Theta(n^{\log_b a})$

2. **Case 2:** $f(n) = \Theta(n^{\log_b a})$

$$\Rightarrow T(n) = \Theta(n^{\log_b a} \log n)$$

3. **Case 3:** $f(n) = \Omega(n^{\log_b a + \epsilon})$, $\epsilon > 0$ and regularity condition

$$af(n/b) \leq cf(n) \text{ for some } c < 1$$

$$\Rightarrow T(n) = \Theta(f(n))$$

Problem (i) $T(n) = 3T(n/2) + n^2$

- Identify parameters:

- $a = 3, b = 2, f(n) = n^2$

- Compute $n^{\log_b a} = n^{\log_2 3} \approx n^{1.58496}$

- Compare $f(n)$ with $n^{\log_2 3}$:

- $n^2 = \Omega(n^{1.58496 + \epsilon})$, $\epsilon \approx 0.415 \rightarrow$ Case 3

- Check regularity: $3(n/2)^2 = 3n^2/4 \leq cn^2$ for $c < 1 \rightarrow$ True

$$\Rightarrow T(n) = \Theta(n^2)$$

Problem (ii) $T(n) = 2T(n/2) + n \log n$

- Identify parameters:

- $a = 2, b = 2, f(n) = n \log n$

- Compute $n^{\log_b a} = n^{\log_2 2} = n^1 = n$

- Compare $f(n)$ with $n^{\log_2 2} = n$:

- $f(n) = n \log n = \Theta(n^{\log_2 2} \log n) \rightarrow$ Case 2

$$\Rightarrow T(n) = \Theta(n \log^2 n)$$

Key Points

- Master theorem simplifies **solving divide-and-conquer recurrences**
 - Provides **tight asymptotic bounds** without expanding recurrence fully
 - Applies to recurrences of the form $T(n) = aT(n/b) + f(n)$
-

Example Explanation

- For $T(n) = 3T(n/2) + n^2$, the recurrence arises in algorithms dividing the problem into 3 subproblems of half size each and performing quadratic work. Master theorem shows total complexity grows like n^2 .
- For $T(n) = 2T(n/2) + n \log n$, the recurrence could represent a merge-type algorithm with slightly higher merge cost. Using Master theorem, the total complexity is $n \log^2 n$.

OR

(c) Write binary search algorithm using recursion. Derive its recurrence relation. Calculate its complexity using master theorem. (7 mark)

Introduction / Definition:

Binary Search is an efficient algorithm to find the **position of a target element** in a **sorted array**. It works on the principle of **divide and conquer**, repeatedly dividing the array into two halves and comparing the target with the middle element. Binary search reduces the search space **logarithmically**, making it very efficient for large datasets.

Recursive Binary Search Algorithm

Algorithm (Pseudocode):

BinarySearch(A, low, high, key):

if low > high:

return -1 // Key not found

mid = (low + high) / 2

if A[mid] == key:

return mid // Key found

else if key < A[mid]:

return BinarySearch(A, low, mid-1, key) // Search left half

else:

return BinarySearch(A, mid+1, high, key) // Search right half

Explanation:

- Start with low = 0 and high = n-1.
- Compute mid = (low + high)/2.
- Compare the middle element with the key.
- Recursively search the **left half** if key is smaller, or the **right half** if key is larger.
- Stop when low > high (element not found) or key is found.

Derivation of Recurrence Relation

- Let $T(n)$ be the time to search in an array of size n .
- Binary search divides the array into **two halves** and searches in **one half only**, plus constant time for comparison:

$$T(n) = T(n/2) + O(1)$$

- Base case: $T(1) = O(1)$
-

Solving Recurrence Using Master Theorem

- Recurrence: $T(n) = T(n/2) + O(1)$
- Compare with Master theorem form: $T(n) = aT(n/b) + f(n)$
 - $a = 1, b = 2, f(n) = O(1)$
- Compute $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$
- Compare $f(n) = O(1) = \Theta(n^0) \rightarrow$ Case 2 of Master Theorem

$$T(n) = \Theta(\log n)$$

Key Features

- Works only on **sorted arrays**
 - Uses **divide-and-conquer strategy**
 - Efficient: **$O(\log n)$ time complexity**
 - Recursive implementation is simple and intuitive
-

Advantages

- Very fast compared to linear search for large arrays
- Minimal comparisons due to halving the search space

Disadvantages

- Requires **sorted array**
 - Recursive implementation uses **stack space**, can cause overhead
-

Applications

- Searching in large datasets (databases, dictionaries)
 - Finding elements in **sorted lists or arrays**
 - Used in **binary search trees, search algorithms, and algorithmic problems**
-

Example Explanation

- Array: [2, 5, 8, 12, 16, 23, 38, 45], key = 16
- Steps:
 1. mid = 3, $A[3] = 12 < 16 \rightarrow$ search right half [16,23,38,45]
 2. mid = 1 (in subarray), $A[1] = 16 \rightarrow$ key found

A diagram would show the array split at each recursive step, highlighting the mid element until the key is found.

Q.3 (a) Explain the steps of greedy strategy for solving a problem. (3 mark)

Greedy Strategy

Introduction / Definition:

The **greedy strategy** is an algorithmic approach in which a solution is built **step by step** by making the **locally optimal choice at each step**, with the hope that these local choices will lead to a globally optimal solution. Greedy algorithms are simple, fast, and easy to implement, and they are commonly used in optimization problems.

Steps of Greedy Strategy

1. **Define the Problem and Objective:**

Clearly identify the problem to be solved and determine the objective function that needs to be optimized (minimized or maximized).

2. **Greedy Choice Property:**

At each step, select the option that appears to be the **best at that moment** without considering future consequences.

3. **Feasibility Check:**

Ensure that the chosen option does not violate the problem constraints and keeps the partial solution valid.

4. **Solution Construction:**

Add the selected choice to the current solution and reduce the problem to a smaller subproblem.

5. **Termination:**

Repeat the above steps until a complete solution is obtained.

Key Points

- Decisions are made **locally**, not globally.
- No backtracking is involved.
- Works only for problems that satisfy **greedy-choice property** and **optimal substructure**.

(b) Write an algorithm for Max-Min problem using divide and conquer approach. Calculate its complexity. (4 mark)

Definition / Introduction:

The **Max–Min problem** is to find the **maximum and minimum elements** from a given set or array of numbers. Using the **divide and conquer approach**, the array is recursively divided into smaller subarrays, and the maximum and minimum are found

efficiently by combining the results. This method reduces the number of comparisons compared to the straightforward approach.

Algorithm (Divide and Conquer)

Pseudocode:

MaxMin(A, low, high):

 if low == high:

 max = min = A[low]

 return (max, min)

 if high == low + 1:

 if A[low] > A[high]:

 max = A[low]

 min = A[high]

 else:

 max = A[high]

 min = A[low]

 return (max, min)

mid = (low + high) / 2

(max1, min1) = MaxMin(A, low, mid)

(max2, min2) = MaxMin(A, mid+1, high)

max = maximum(max1, max2)

```
min = minimum(min1, min2)
return (max, min)
```

Explanation of Algorithm

- If the array has **one element**, it is both minimum and maximum.
 - If the array has **two elements**, one comparison is used to find min and max.
 - For more than two elements, the array is divided into two halves.
 - The algorithm recursively finds max and min in both halves.
 - The final maximum and minimum are obtained by comparing the results of both halves.
-

Time Complexity

- Let $T(n)$ be the time complexity for n elements.
 - Recurrence relation:
 -
 - Using Master Theorem:
 - $a = 2, b = 2, f(n) = O(1)$
 - $n^{\log_b a} = n$
 - **Time Complexity:** $\Theta(n)$
-

Key Points

- Uses fewer comparisons than simple linear approach.

- Efficient for large datasets.
- Based on **divide and conquer strategy**.

(c) The matrices A(5X10), B(10X15), C(15X20), and D(20X25) are given. Solve the matrix chain multiplication problem using dynamic programming. (7 mark)

Introduction / Definition:

The **Matrix Chain Multiplication (MCM)** problem deals with finding the **most efficient way to multiply a sequence of matrices**. Since matrix multiplication is associative, the order of multiplication can be changed, but different parenthesizations result in different numbers of scalar multiplications. Dynamic programming is used to find the **optimal order** that minimizes the total number of scalar multiplications.

Given Matrices

- $A = 5 \times 10$
- $B = 10 \times 15$
- $C = 15 \times 20$
- $D = 20 \times 25$

Let the dimension array be:

$$p = [5, 10, 15, 20, 25]$$

Number of matrices $n = 4$.

Dynamic Programming Approach

- Define $m[i][j]$ as the **minimum number of scalar multiplications** needed to compute the product of matrices A_i to A_j .
- If $i = j$, then $m[i][j] = 0$.
- Recurrence relation:

$$m[i][j] = \min_{i \leq k < j} \{m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j\}$$

Step-by-Step Computation

Step 1: Length = 2

- $m[1,2] = 5 \times 10 \times 15 = 750$
- $m[2,3] = 10 \times 15 \times 20 = 3000$
- $m[3,4] = 15 \times 20 \times 25 = 7500$

Step 2: Length = 3

- $m[1,3]$:
 - $k = 1: 0 + 3000 + (5 \times 10 \times 20) = 4000$
 -
 -
 - **Minimum = 2250**
 - $m[2,4]$:
 - $k = 2: 0 + 7500 + (10 \times 15 \times 25) = 11250$
 - $k = 3: 3000 + 0 + (10 \times 20 \times 25) = 8000$
 - **Minimum = 8000**
 - **Step 3: Length = 4**
 - $m[1,4]$:
 - $k = 1: 0 + 8000 + (5 \times 10 \times 25) = 9250$

- $k = 2: 750 + 7500 + (5 \times 15 \times 25) = 10125$

- $k = 3: 2250 + 0 + (5 \times 20 \times 25) = 4750$

- **Minimum = 4750**

- **Optimal Solution**

- **Minimum number of scalar multiplications:**

- $\boxed{4750}$

- **Optimal Parenthesization:**

- $\boxed{((A(BC))D)}$

- ---

- **Key Features**

- Uses overlapping subproblems and optimal substructure

- Reduces exponential complexity to polynomial

- Guarantees minimum computation cost

- ---

- **Time and Space Complexity**

- **Time Complexity:** $O(n^3)$

- **Space Complexity:** $O(n^2)$

- ---

- **Example Explanation**

- Multiplying B and C first reduces intermediate matrix size.

- The result is then multiplied with A , and finally with D .

- A diagram would show matrices grouped as $(A(BC))D$, highlighting reduced scalar operations

OR

Q.3 (a) Enlist the general characteristics of greedy algorithms. (3 mark)

Introduction / Definition:

A **greedy algorithm** is an algorithmic technique that constructs a solution **step by step** by choosing the **locally optimal choice** at each step, with the aim of obtaining a globally optimal solution. Greedy algorithms are simple, fast, and widely used in optimization problems.

General Characteristics

1. Greedy Choice Property:

At every step, the algorithm selects the option that seems **best at that moment** without considering future consequences.

2. Optimal Substructure:

The problem can be solved optimally by combining **optimal solutions of its subproblems**.

3. Incremental Solution Construction:

The solution is built **gradually**, adding one element at a time.

4. No Backtracking:

Once a choice is made, it is **never reconsidered or changed**.

5. Simple and Efficient:

Greedy algorithms are easy to implement and generally have **low time complexity**.

(b) Write an algorithm for matrix multiplication using divide and conquer approach. Calculate its complexity. (4 mark)

Introduction / Definition:

Matrix multiplication is a fundamental operation in computer science and mathematics. The **divide and conquer approach** for matrix multiplication works by dividing large matrices into smaller submatrices, recursively multiplying them, and then combining the results. This approach forms the basis of advanced algorithms like **Strassen's matrix multiplication**.

Algorithm (Divide and Conquer)

Pseudocode:

MatrixMultiply(A, B, n):

 if n == 1:

$C[1][1] = A[1][1] * B[1][1]$

 return C

Divide A into A11, A12, A21, A22

Divide B into B11, B12, B21, B22

$C11 = \text{Multiply}(A11, B11) + \text{Multiply}(A12, B21)$

$C12 = \text{Multiply}(A11, B12) + \text{Multiply}(A12, B22)$

$C21 = \text{Multiply}(A21, B11) + \text{Multiply}(A22, B21)$

$C22 = \text{Multiply}(A21, B12) + \text{Multiply}(A22, B22)$

Combine C11, C12, C21, C22 into matrix C

return C

Explanation of Algorithm

- If the matrix size is 1×1 , directly multiply the elements.
- Otherwise, divide both matrices into **four equal submatrices**.
- Recursively multiply the corresponding submatrices.
- Add the intermediate results to form the final result matrix.
- Continue until the base case is reached.

Time Complexity

- Let $T(n)$ be the time to multiply two $n \times n$ matrices.
- Recurrence relation:

$$T(n) = 8T(n/2) + O(n^2)$$

- Using Master Theorem:
 - $a = 8, b = 2, f(n) = O(n^2)$
 - $n^{\log_b a} = n^{\log_2 8} = n^3$
- **Time Complexity:**

$$T(n) = \Theta(n^3)$$

Key Points

- Uses recursive division of matrices.
- Reduces problem size at each step.
- Foundation for optimized algorithms like Strassen's method.

(c) For the given set of items and knapsack capacity = 5 kg, find the optimal solution for the 0/1 knapsack problem using dynamic programming.

knapsack problem using dynamic

Item	Weight	Value
I1	2	3
I2	3	4
I3	4	5
I4	5	6

Introduction / Definition

The **0/1 Knapsack problem** is a classic optimization problem in which we are given a set of items, each having a **weight** and a **value**, and a knapsack with a fixed **capacity**. The objective is to **maximize the total value** of items placed in the knapsack such that the total weight does not exceed the knapsack capacity.

In the **0/1 knapsack**, each item can be either **selected (1)** or **not selected (0)**; fractional selection is not allowed.

Dynamic programming is used to solve this problem efficiently by breaking it into **overlapping subproblems** and storing their solutions.

Given Data

Item Weight (kg) Value

I1 2 3

I2 3 4

I3 4 5

I4 5 6

- **Knapsack Capacity (W) = 5 kg**
- **Number of items (n) = 4**

Dynamic Programming Formulation

Let

$$DP[i][w]$$

= Maximum value obtained using first i items with capacity w

Recurrence Relation:

- If item i is **not included**:

$$DP[i][w] = DP[i - 1][w]$$

- If item i is **included** (when $\text{weight} \leq \text{capacity}$):

Base Condition:

- $DP[0][w] = 0$ for all w
- $DP[i][0] = 0$ for all i

DP Table Construction

Capacity $W = 5$

Items ↓ / Weight →	0	1	2	3	4	5
0 items	0	0	0	0	0	0
I1 (2,3)	0	0	3	3	3	3
I2 (3,4)	0	0	3	4	4	7
I3 (4,5)	0	0	3	4	5	7
I4 (5,6)	0	0	3	4	5	7

Optimal Solution

- **Maximum value = 7**
- Achieved by selecting:
 - **Item I1 (weight 2, value 3)**
 - **Item I2 (weight 3, value 4)**

Total Weight = 2 + 3 = 5 kg

Total Value = 3 + 4 = 7

Key Features of Dynamic Programming Approach

- Uses **optimal substructure**
 - Avoids recomputation using table storage
 - Guarantees **optimal solution**
-

Time and Space Complexity

- **Time Complexity:** $O(nW)$
- **Space Complexity:** $O(nW)$

Where:

- n = number of items
 - W = knapsack capacity
-

Example Explanation (in words)

The algorithm evaluates each item one by one and checks whether including it increases the total value without exceeding the knapsack capacity. A table is built where each cell stores the best

possible value for a given number of items and capacity. The final answer is obtained from the last cell of the table.

A diagram would show a table with rows as items and columns as capacities, filled step by step.

Q.4 (a) Explain DFS with suitable example. (3 mark)

Introduction / Definition:

Depth First Search (DFS) is a graph traversal technique used to explore all the vertices of a graph. In DFS, the traversal starts from a selected source vertex and explores **as far as possible along one branch before backtracking**. It uses a **stack-based approach**, either explicitly using a stack or implicitly using **recursion**.

Explanation of DFS Algorithm

- Start from a given **source vertex** and mark it as visited.
- Visit one of its adjacent (unvisited) vertices.
- Continue moving deeper in the graph until a vertex with no unvisited adjacent vertices is found.
- Backtrack to the previous vertex and explore other unvisited neighbors.
- Repeat this process until **all vertices** of the graph are visited.

DFS can be implemented using **recursion** or an **explicit stack**.

DFS Algorithm (Steps)

1. Select a starting vertex and mark it as visited.
2. Visit an adjacent unvisited vertex.

3. Recursively apply DFS on that vertex.
4. If no unvisited adjacent vertices are available, backtrack.
5. Continue until all vertices are visited.

Suitable Example (Explanation in Words)

Consider a graph with vertices **A, B, C, D, and E** and edges connecting them as follows:

- A is connected to B and C
- B is connected to D
- C is connected to E

DFS Traversal starting from A:

- Start at A → visit B → visit D
- D has no unvisited neighbors → backtrack to B
- Backtrack to A → visit C → visit E

DFS Order:

$$A \rightarrow B \rightarrow D \rightarrow C \rightarrow E$$

A diagram would show a graph where traversal goes deep along one path before returning to explore other paths.

Key Features

- Uses **stack or recursion**
- Traverses deep before exploring breadth
- Works for both **directed and undirected graphs**

Applications

- Finding connected components
 - Topological sorting
 - Detecting cycles in graphs
-

(b) Enlist the advantages and disadvantages of dynamic programming. (4 mark)

Dynamic programming (DP) is an algorithm design technique used to solve complex problems by breaking them into **overlapping subproblems** and storing their results to avoid repeated computations. It is widely used in optimization problems where an optimal solution can be built from optimal solutions of smaller subproblems.

Advantages of Dynamic Programming

- **Avoids Re-computation:**
Stores the result of subproblems, which prevents solving the same problem repeatedly.
- **Improves Time Efficiency:**
Reduces exponential-time recursive solutions to **polynomial time** in many cases.
- **Guarantees Optimal Solution:**
Works well for problems having **optimal substructure** and overlapping subproblems.
- **Systematic Approach:**
Provides a clear, step-by-step method to solve complex problems.

Disadvantages of Dynamic Programming

- **High Space Requirement:**
Requires additional memory to store DP tables, leading to high space complexity.
- **Complex Implementation:**
Designing DP solutions and defining correct states and transitions can be difficult.
- **Not Always Applicable:**
Works only for problems with optimal substructure and overlapping subproblems.
- **Initialization Overhead:**
Filling large tables may cause overhead for problems with large input sizes.

(c) Define minimum spanning tree. Explain Prim's algorithm with suitable example. (7 mark)

Minimum Spanning Tree and Prim's Algorithm

Definition: Minimum Spanning Tree (MST)

A **Minimum Spanning Tree (MST)** of a connected, undirected, weighted graph is a subgraph that:

- Includes **all the vertices** of the graph
- Contains **exactly $(n - 1)$ edges**, where n is the number of vertices
- Has **no cycles**
- Has the **minimum possible total edge weight**

The main objective of an MST is to connect all vertices using the **least total cost**.

Introduction to Prim's Algorithm

Prim's algorithm is a **greedy algorithm** used to find the Minimum Spanning Tree of a graph. It starts with a single vertex and repeatedly adds the **minimum weight edge** that connects a vertex in the MST to a vertex outside the MST.

Prim's algorithm always maintains a **growing tree** and expands it until all vertices are included.

Steps of Prim's Algorithm

1. Select any vertex as the **starting vertex**.
 2. Initialize a set that contains vertices already included in the MST.
 3. From all edges connecting the MST set to remaining vertices, choose the **edge with minimum weight**.
 4. Add the selected edge and the corresponding vertex to the MST.
 5. Repeat steps 3 and 4 until all vertices are included.
 6. The resulting tree is the **Minimum Spanning Tree**.
-

Suitable Example (Explained in Words)

Consider a graph with vertices **A, B, C, and D** and weighted edges:

- A–B with weight 1
- A–C with weight 3
- B–C with weight 2
- B–D with weight 4

- C–D with weight 5

Applying Prim's Algorithm:

- Start from vertex A
- Select edge A–B (weight 1) → add B
- From available edges, select B–C (weight 2) → add C
- Next minimum edge is B–D (weight 4) → add D

Now all vertices are included.

The MST contains edges **A–B, B–C, and B–D**.

A diagram would show a connected graph where the selected minimum edges form a tree without cycles.

Key Features of Prim's Algorithm

- Based on **greedy strategy**
- Builds MST **vertex by vertex**
- Works only for **connected graphs**
- Ensures minimum total edge weight

Advantages of Prim's Algorithm

- Simple and easy to understand
- Efficient for **dense graphs**
- Guarantees optimal solution

Disadvantages of Prim's Algorithm

- Not suitable for **disconnected graphs**

- Requires priority queue for efficiency
 - Slightly complex implementation compared to Kruskal's algorithm
-

Applications of Minimum Spanning Tree

- Network design (computer networks, electrical grids)
- Road and railway planning
- Cost optimization problems
- Clustering in data analysis

OR

Q.4 (a) Explain BFS with suitable example. (3 mark)

Breadth First Search (BFS)

Introduction / Definition:

Breadth First Search (BFS) is a graph traversal algorithm used to explore all the vertices of a graph. In BFS, traversal starts from a given source vertex and visits **all adjacent vertices first** before moving to the next level of vertices. BFS follows a **level-by-level traversal** approach and uses a **queue** data structure.

Explanation of BFS:

In BFS, the source vertex is first visited and marked as visited. All its adjacent vertices are then inserted into a queue. Each vertex is removed from the queue one by one, visited, and its unvisited adjacent vertices are added to the queue. This process continues until all vertices of the graph are visited.

Suitable Example (Explained in Words):

Consider a graph with vertices **A, B, C, D, and E**. Let vertex **A** be the starting point. First, A is visited, then its adjacent vertices B and C are visited. Next, the neighbors of B and C, such as D and E, are visited. The BFS traversal order is:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$$

A diagram would show vertices arranged in levels, where all vertices at one level are visited before moving to the next level.

(b) Define principal of optimality. Explain its use in Dynamic Programming Method. (3 mark)

Principle of Optimality in Dynamic Programming

Definition

The **principle of optimality** states that *an optimal solution of a problem contains within it optimal solutions to its subproblems*. In other words, regardless of the initial choice, the remaining decisions must form an optimal solution for the remaining subproblem.

This principle is the **foundation of dynamic programming**.

Explanation of the Principle of Optimality

- If a problem can be broken into smaller subproblems, and
- The optimal solution of the whole problem depends on optimal solutions of these subproblems,
- Then the problem satisfies the **principle of optimality**.

Dynamic programming uses this principle to ensure that solving and storing subproblem solutions leads to an optimal final solution.

Use of Principle of Optimality in Dynamic Programming

- It allows the problem to be **divided into stages** or states.
 - Ensures that solving each subproblem **once** is sufficient.
 - Makes it possible to apply **bottom-up or top-down** DP approaches.
 - Guarantees that the final result obtained from stored subproblem solutions is optimal.
-

Example (Explained in Words)

In the **shortest path problem**, if the shortest path from A to D goes through B, then the path from A to B must also be the shortest. If it were not optimal, the overall path could not be optimal. This clearly follows the principle of optimality.

(c) "A greedy strategy will work for fractional Knapsack problem but not for 0/1", is this true or false? Explain with suitable example. (7 mark)

Greedy Strategy for Fractional and 0/1 Knapsack Problem

Statement

“A greedy strategy works for the Fractional Knapsack problem but not for the 0/1 Knapsack problem.”

This statement is **TRUE**.

Introduction

The **Knapsack problem** is a classic optimization problem where a set of items, each with a weight and profit, must be selected to maximize total profit without exceeding the knapsack capacity. There are two main types:

- **Fractional Knapsack**
- **0/1 Knapsack**

The applicability of the greedy strategy differs for these two problems.

Greedy Strategy in Fractional Knapsack

In the **fractional knapsack problem**, items can be **broken into smaller fractions**. The greedy approach selects items based on the **highest profit-to-weight ratio (p/w)**.

Why Greedy Works:

- Items can be taken **partially**.
 - Local optimal choice (highest p/w) always leads to a **global optimal solution**.
 - The problem satisfies the **greedy choice property**.
-

Example: Fractional Knapsack

Item Profit Weight p/w

A 60 10 6

B 100 20 5

C 120 30 4

Knapsack capacity = 50

Greedy selection (by p/w):

- Take A completely (10 kg)
- Take B completely (20 kg)

- Take **2/3 of C** (20 kg)

This gives the **maximum profit**, proving greedy strategy works.

Greedy Strategy in 0/1 Knapsack

In the **0/1 knapsack problem**, items are either **fully selected or rejected**. Fractional selection is **not allowed**.

Why Greedy Fails:

- Choosing locally optimal items may block better global combinations.
 - The problem does **not satisfy greedy choice property**.
 - Optimal solution depends on **combinations**, not single best choices.
-

Example: 0/1 Knapsack (Greedy Failure)

Item Profit Weight

A 60 10

B 100 20

C 120 30

Knapsack capacity = 50

Greedy choice by p/w:

- Select A (10 kg)
- Select B (20 kg)
- Cannot select C (exceeds capacity)

Total profit = **160**

Optimal solution:

- Select B and C
Total profit = **220**

Thus, greedy strategy fails to find the optimal solution.

Key Comparison

Aspect	Fractional Knapsack	0/1 Knapsack
Item division	Allowed	Not allowed
Greedy applicable	Yes	No
Optimality guarantee	Always	Not guaranteed
Solution method	Greedy	Dynamic Programming

Q.5 (a) Define string matching problem. Define valid shift and invalid shift. (3 mark)

The **string matching problem** involves finding all occurrences of a **pattern string (P)** within a **given text string (T)**. The objective is to determine the positions in the text where the pattern exactly matches a substring of the text. String matching is a fundamental problem in computer science and is widely used in text processing, information retrieval, and bioinformatics.

Valid Shift

A **valid shift** occurs when the pattern is aligned with the text in such a way that **all characters of the pattern match exactly** with the corresponding characters of the text. At a valid shift, the pattern is said to be successfully found in the text.

Invalid Shift

An **invalid shift** occurs when **at least one character** of the pattern does not match the corresponding character in the text during alignment. In such a case, the pattern is shifted to the next position in the text for further comparison.

(b) Define the followings: i) Articulation point ii) Acyclic Directed Graph iii) Back Edge iv) Tree (4 mark)

i) Articulation Point

An **articulation point** (or cut vertex) is a vertex in a graph whose **removal along with its incident edges increases the number of connected components** of the graph. In other words, removing this vertex makes the graph disconnected. Articulation points are important in network reliability and graph connectivity analysis.

ii) Acyclic Directed Graph (DAG)

An **Acyclic Directed Graph (DAG)** is a **directed graph that contains no directed cycles**. This means there is no path that starts and ends at the same vertex following the direction of edges. DAGs are widely used in task scheduling, dependency resolution, and topological sorting.

iii) Back Edge

A **back edge** is an edge in a directed graph that **connects a vertex to one of its ancestors** in a depth first search (DFS) traversal. The presence of a back edge indicates the existence of a **cycle** in the graph.

iv) Tree

A **tree** is a connected, undirected graph that contains **no cycles**. For a tree with n vertices, there are exactly **$(n - 1)$ edges**. Trees are hierarchical structures commonly used in file systems, data organization, and network routing.

(c) Explain P, NP, NP complete and NP-Hard problems. Give examples of each. (7 mark)

Introduction

In computational complexity theory, problems are classified based on the **time required to solve them** and **verify their solutions**. The most important complexity classes are **P, NP, NP-Complete, and NP-Hard**. These classifications help in understanding the difficulty of computational problems and deciding appropriate solution strategies.

1. Class P Problems

Definition

A problem belongs to **Class P (Polynomial time)** if it can be **solved in polynomial time** using a deterministic algorithm.

Explanation

- The solution can be found efficiently.
- Time complexity is of the form $O(n^k)$, where k is a constant.
- These problems are considered **tractable** or easy to solve.

Examples

- Linear Search
- Binary Search
- Sorting algorithms (Merge Sort, Bubble Sort)

- Shortest path problem using Dijkstra's algorithm (without negative edges)
-

2. Class NP Problems

Definition

A problem belongs to **Class NP (Non-deterministic Polynomial time)** if a proposed solution can be **verified in polynomial time**, even if finding the solution may be difficult.

Explanation

- The solution may not be easy to compute.
- However, once a solution is given, it can be checked efficiently.
- All **P problems are also NP problems**, but not all NP problems are P.

Examples

- Hamiltonian Path problem
 - Boolean Satisfiability Problem (SAT)
 - Subset Sum problem
-

3. NP-Complete Problems

Definition

A problem is **NP-Complete** if:

1. It belongs to **NP**, and
2. Every problem in NP can be **polynomially reduced** to it.

Explanation

- NP-Complete problems are the **hardest problems in NP**.

- If any NP-Complete problem is solved in polynomial time, then **$P = NP$** .
- No polynomial-time algorithm is known for NP-Complete problems.

Examples

- Traveling Salesman Problem (decision version)
 - 0/1 Knapsack problem
 - Hamiltonian Cycle problem
 - SAT problem
-

4. NP-Hard Problems

Definition

A problem is **NP-Hard** if it is **at least as hard as NP-Complete problems**, but it **may not belong to NP**.

Explanation

- The solution does not need to be verifiable in polynomial time.
- NP-Hard problems may be **optimization or undecidable problems**.
- All NP-Complete problems are NP-Hard, but not all NP-Hard problems are NP-Complete.

Examples

- Traveling Salesman Problem (optimization version)
 - Halting Problem
 - Job Scheduling optimization problems
-

Comparison Summary

Class	Solvable in Polynomial Time	Verifiable in Polynomial Time
P	Yes	Yes
NP	No (in general)	Yes
NP-Complete	No	Yes
NP-Hard	Not necessarily	Not necessarily

Conclusion

- **P problems** are easy to solve.
- **NP problems** are easy to verify.
- **NP-Complete problems** are the hardest problems in NP.
- **NP-Hard problems** are even more difficult and may not be verifiable efficiently.

Understanding these classes is essential for selecting efficient algorithms and identifying problems that require approximation or heuristic approaches.

OR

Q.5 (a) Explain Backtracking Method. Give solutions for 4-queens problem using backtracking method. (3 mark)

Backtracking Method and 4-Queens Problem

Backtracking Method

Backtracking is an algorithmic technique used to solve problems by **building the solution step by step** and removing those solutions that

fail to satisfy the problem constraints. It systematically explores all possible options and **abandons a path as soon as it determines that the path cannot lead to a valid solution.**

Backtracking is mainly used for **constraint satisfaction problems** such as N-Queens, graph coloring, and Sudoku.

Backtracking Approach for 4-Queens Problem

The **4-Queens problem** involves placing **four queens on a 4×4 chessboard** such that no two queens attack each other. This means:

- No two queens can be in the same **row**
- No two queens can be in the same **column**
- No two queens can be on the same **diagonal**

Using backtracking, queens are placed **row by row**. For each row, all columns are checked. If placing a queen in a column violates any constraint, that position is rejected, and the algorithm **backtracks** to try another column.

Solution for 4-Queens Problem

Using backtracking, the **valid solutions** for the 4-Queens problem are:

- Queen positions: (1,2), (2,4), (3,1), (4,3)
- Queen positions: (1,3), (2,1), (3,4), (4,2)

A diagram would show queens placed such that none attack each other horizontally, vertically, or diagonally.

(b) Explain in-order, pre-order and post-order traversals of the graph.
(4 mark)

Definition:

Tree traversal is the process of visiting all the nodes in a tree data structure in a systematic way. Traversals help in searching, sorting, and performing operations on tree nodes. There are three main types of depth-first traversals: in-order, pre-order, and post-order.

1. In-Order Traversal:

- In in-order traversal, nodes are visited in the **Left → Root → Right** sequence.
- First, the left subtree of a node is visited, then the node itself, and finally the right subtree.
- **Purpose:** It is primarily used for **binary search trees (BSTs)** because it visits nodes in **ascending order**.
- **Example:** For a BST with nodes 10, 20, 30 where 20 is root, 10 left child, 30 right child → traversal order: 10, 20, 30.

2. Pre-Order Traversal:

- In pre-order traversal, nodes are visited in the **Root → Left → Right** sequence.
- The root node is visited first, then the left subtree, and finally the right subtree.
- **Purpose:** Useful for **copying trees** and generating prefix expressions in expression trees.
- **Example:** For the same tree above → traversal order: 20, 10, 30.

3. Post-Order Traversal:

- In post-order traversal, nodes are visited in the **Left → Right → Root** sequence.

- First, the left subtree is visited, then the right subtree, and finally the root node.
- **Purpose:** Often used for **deleting a tree** or evaluating postfix expressions.
- **Example:** For the same tree above → traversal order: 10, 30, 20.

Key Points / Features:

- All three are **depth-first traversals**, meaning they go deep into one subtree before visiting another.
- In-order gives **sorted output** for BSTs.
- Pre-order helps in **replicating tree structure**.
- Post-order is useful for **clean-up and evaluation** tasks.

Advantages:

- Systematic visiting ensures no node is missed.
- Different traversals serve different computational purposes.

Disadvantages:

- For large trees, recursive implementation may cause stack overflow; iterative methods may be needed.

Applications:

- Expression evaluation in compilers.
- Searching and sorting in trees.
- Copying and deleting tree structures.

(c) Write short note on the followings: i) Approximation algorithms ii) Randomized algorithms (7 mark)

1. Approximation Algorithms

Definition:

Approximation algorithms are algorithms used for **NP-hard or NP-complete problems**, where finding the exact optimal solution is computationally infeasible. Instead, these algorithms provide a **solution that is close to optimal** within a guaranteed bound.

Detailed Explanation:

- These algorithms are designed for problems where exact solutions require **exponential time**.
- They aim to produce solutions efficiently (in **polynomial time**) with a **known approximation ratio**.
- The **approximation ratio** is the measure of how close the solution is to the optimal one.

Key Features:

- Efficient and practical for large instances of hard problems.
- Do not guarantee the best solution, but provide a **good-enough solution**.
- Often involve **heuristics, greedy methods, or local search** techniques.

Advantages:

- Reduces computation time drastically.
- Useful in real-world applications where approximate solutions are acceptable.

Disadvantages:

- The solution may not be exact.
- Quality depends on the design and problem constraints.

Applications:

- **Traveling Salesman Problem (TSP):** Using nearest-neighbor heuristic.
- **Knapsack Problem:** Using greedy or fractional methods.
- **Vertex Cover Problem:** Provides near-optimal coverage in graphs.

Example:

For TSP with cities A, B, C, D, exact solution may take long; approximation algorithm gives a route slightly longer than optimal but computed quickly.

2. Randomized Algorithms

Definition:

Randomized algorithms are algorithms that use **random numbers or random choices** in their logic to make decisions during execution. The randomness can influence the **efficiency** or **output** of the algorithm.

Detailed Explanation:

- These algorithms are classified based on the **type of randomness**:
 - **Las Vegas algorithms:** Always produce correct results; running time may vary.
 - **Monte Carlo algorithms:** May produce incorrect results with a small probability, but usually fast.
- They are often used for problems where deterministic solutions are slow or complex.

Key Features:

- Use random numbers or probabilistic decisions.

- Can simplify complex problems and improve performance.
- Probabilistic guarantees rather than deterministic ones.

Advantages:

- Faster and simpler than deterministic counterparts.
- Can handle large datasets efficiently.
- Useful in online and real-time applications.

Disadvantages:

- May produce different results on different runs (Monte Carlo type).
- Hard to debug due to inherent randomness.

Applications:

- **Primality testing:** Miller-Rabin algorithm.
- **Hashing:** Randomized hash functions reduce collisions.
- **Graph algorithms:** Randomized algorithms for minimum cut and connectivity.

Example:

In primality testing, Miller-Rabin algorithm randomly selects numbers to test if a large number is prime. It is **fast** and highly **probabilistically accurate**, though not always guaranteed deterministically.